NAME: GUGGILLA PALANI KUMAR REDDY

USN: 23BTREC035

DEPARTMENT: ECE(ELECTRONICS AND COMMUNICATION ENGINEERING)

HCL ID :

SECTION A: SQL Question 1: Database Design and Advanced Queries (8 Marks)

Tasks:

a. Display employee names, department names, and manager names using joins. (2 marks

```
49
50 •  SELECT e.EmployeeName,
51           d.DepartmentName,
52           m.EmployeeName AS ManagerName
53    FROM Employees e
54    LEFT JOIN Departments d
55        ON e.DepartmentID = d.DepartmentID
56    LEFT JOIN Employees m
57        ON e.ManagerID = m.EmployeeID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeName	DepartmentName	ManagerName
Arun	IT	NULL
Bharath	IT	Arun
Charan	IT	Arun
Deepak	IT	Bharath
Eshwar	HR	NULL
Farhan	HR	Eshwar
Ganesh	HR	Eshwar
Harish	Finance	NULL
Ishan	Finance	Harish
Jay	Finance	Harish
Kiran	Marketing	NULL

Result 1

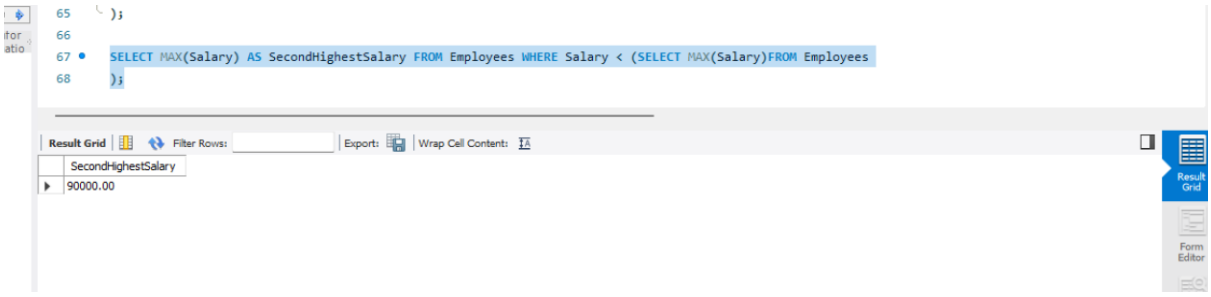
b. Find employees earning more than their department's average salary. (2 marks

```
58
59  SELECT EmployeeName, DepartmentID, Salary
60  FROM Employees e
61  WHERE Salary > (
62      SELECT AVG(Salary)
63      FROM Employees
64      WHERE DepartmentID = e.DepartmentID
65  );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeName	DepartmentID	Salary
Arun	1	90000.00
Bharath	1	70000.00
Eshwar	2	80000.00
Harish	3	95000.00
Kiran	4	75000.00

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)



d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)

```
WITH RankedEmployees AS (
    SELECT EmployeeID, EmployeeName,DepartmentID, Salary,DENSE_RANK() OVER (
    PARTITION BY DepartmentID
                ORDER BY Salary DESC
            ) AS SalaryRank
    FROM Employees
)
SELECT *
FROM RankedEmployees
WHERE SalaryRank <= 3
ORDER BY DepartmentID, Salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	EmployeeName	DepartmentID	Salary	SalaryRank
1	Arun	1	90000.00	1
2	Bharath	1	70000.00	2
3	Charan	1	60000.00	3
4	Deepak	1	60000.00	3
5	Eshwar	2	80000.00	1
6	Farhan	2	50000.00	2
15	Praveen	2	50000.00	2
7	Ganesh	2	45000.00	3
8	Harish	3	95000.00	1
9	Ishan	3	65000.00	2
10	Jay	3	55000.00	3
11	Kiran	4	75000.00	1
12	Lokesh	4	55000.00	2
13	Manoj	4	40000.00	3

Question 2: Create a Sales table with sale id ,cutomer id ,sale date , product name ,quantity ,unitprice

Task:

a.Calculate total revenue for each product. (2 marks)

```
SELECT ProductName, SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	TotalRevenue
Laptop	350000.00
Mouse	14500.00
Keyboard	13500.00
Monitor	72000.00

b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)

```sql
WITH MonthlySales AS (
    SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS SaleMonth,
           SUM(Quantity * UnitPrice) AS Revenue
    FROM Sales
    GROUP BY DATE_FORMAT(SaleDate, '%Y-%m')
),
RevenueChanges AS (
    SELECT SaleMonth,
           Revenue,
           LAG(Revenue) OVER (
               ORDER BY SaleMonth
           ) AS PreviousRevenue
    FROM MonthlySales
)
SELECT SaleMonth, Revenue, PreviousRevenue,
       Revenue - PreviousRevenue AS RevenueChange
FROM RevenueChanges
ORDER BY SaleMonth;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SaleMonth	Revenue	PreviousRevenue	RevenueChange
2025-01	107000.00	NULL	NULL
2025-02	79000.00	107000.00	-28000.00
2025-03	68000.00	79000.00	-11000.00
2025-04	107000.00	68000.00	39000.00
2025-05	89000.00	107000.00	-18000.00

Result 9

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)

```sql
ORDER BY SaleMonth;

SELECT *
FROM Employees
WHERE JoiningDate >= '2021-03-15'
  AND JoiningDate < '2022-11-05';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

EmployeeID	EmployeeName	DepartmentID	Salary	JoiningDate	ManagerID
2	Bharath	1	70000.00	2021-03-15	1
3	Charan	1	60000.00	2022-05-20	1
6	Farhan	2	50000.00	2021-08-10	5
12	Lokesh	4	55000.00	2022-01-16	11

Question 3: A company maintains millions of employee and transaction records. Its reporting queries are running slowly

TASK:

(A).An execution plan shows how the database executes a query.

- It identifies table scans, index scans, joins and sorting operations.
- It provides estimated costs and row counts.
- It helps locate slow operations and unnecessary full table scans.
- Developers can improve performance by adding appropriate indexes or rewriting queries.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 marks)

```
135
136 •      CREATE INDEX idx_employee_dept_salary
137      ON Employees (DepartmentID, Salary);
```

- Creating index strategy for the filtering employees by the department id and the salary I would like to sugget the simplest query to by using the “ON” where it can compare the employees by comparing the salary and the departmentID.

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. (2 marks)

- I would like to recommend the query which identify the duplicate customer email address and display the address occurring only once
- I’ll be using the count and having where it is used to filter the email address for
- And ill use GROUP BY to comparing and filtering the email address from the customers table and the values

  example:

  SELECT Email, COUNT(*) FROM Customers GROUP BY Email HAVING COUNT(*) > 1;

SECTION B: LINUX

Question 4: Log Analysis (5 Marks)

- FIRST create the file and inside the file add the data and error lines if the file is not created already
- After creating the file

a. Find all lines containing ERROR. (1 mark)

- grep 'ERROR' "$LOG" using this command helps to find the error lines in the file which is /var/log/application.log

```
[ec2-user@ip-172-31-45-35 ~]$ grep 'ERROR' "$LOG"
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR database-service Connection failed
2026-10-28 11:30:45 ERROR payment-service Timeout occurred
2026-10-29 09:15:00 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR database-service Connection failed
```

b. Display the 20 most recent error entries in the file. (1 mark)

- grep 'ERROR' "$LOG" | tail -n 20 Tail is used to display the last lines of the file here I jave inserted file few error log file show it is displaying how many logs ive insterted it is showing them

```
[ec2-user@ip-172-31-45-35 ~]$ grep 'ERROR' "$LOG" | tail -n 20
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR database-service Connection failed
2026-10-28 11:30:45 ERROR payment-service Timeout occurred
2026-10-29 09:15:00 ERROR auth-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR database-service Connection failed
2026-10-28 11:30:45 ERROR payment-service Timeout occurred
2026-10-29 09:15:00 ERROR auth-service Login failed
```

c. Count the total number of error entries. (1 mark)

- grep -c 'ERROR' "$LOG"

```
[ec2-user@ip-172-31-45-35 ~]$ grep -c 'ERROR' "$LOG"
8
```

d. Extract error entries for a specified date into a separate file. (1 mark)

- grep '^2026-10-28 .*ERROR' "$LOG" > ~/errors_2026-10-28.log cat ~/errors_2026-10-28.log we are saving the file bbythe date of 2026-10-28 and the cat is used to display the saved error log files ive saved it twice it is showing 8

```
[ec2-user@ip-172-31-45-35 ~]$ grep '^2026-10-28 .*ERROR' "$LOG" > ~/errors_2026-10-28.log
cat ~/errors_2026-10-28.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR database-service Connection failed
2026-10-28 11:30:45 ERROR payment-service Timeout occurred
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR database-service Connection failed
2026-10-28 11:30:45 ERROR payment-service Timeout occurred
```

e.Explain how you would calculate the number of errors occurring in each hour. (1 mark)

- awk filters error entries, extracts the hour from the timestamp, and counts errors per hour. sort displays the results in hour order.

Question 5: A Linux server hosts an application that processes confidential financial reports. Perform the following tasks:

a. Create a group named finance and a report file, assuming you have the required privileges. (1 mark)

- sudo groupadd finance sudo touch report.txt using these command to creating the the report finance in the group finance and sudo is used to execute the command

```
[ec2-user@ip-172-31-45-35 ~]$ sudo groupadd finance
sudo touch report.txtsudo groupadd finance
sudo touch report.txt
[ec2-user@ip-172-31-45-35 ~]$ sudo groupadd finance
sudo touch report.txt
```

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. (1 mark)

```
[ec2-user@ip-172-31-45-35 ~]$ sudo chmod 640 report.txt
[ec2-user@ip-172-31-45-35 ~]$ ls -l report.txt
-rw-r-----. 1 ec2-user finance 0 Oct 28 06:23 report.txt
```

c.Identify the processes consuming the most CPU. (1 mark)

- using top command

```
top - 06:32:05 up 13 days,  1:58,  1 user,  load average: 0.00, 0.00, 0.00
Tasks: 106 total,   1 running, 105 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 98.8 id,  0.0 wa,  0.0 hi,  0.0 si,  1.2 st
MiB Mem :    913.1 total,    200.4 free,    198.5 used,    514.2 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    575.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0  174344  18276  11236 S   0.0   2.0   1:11.52 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.23 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:02.16 kworker/0:0H-kblockd
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:03.86 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:18.20 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.44 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:04.65 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:04.63 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:03.65 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:02.43 kworker/1:0H-kblockd
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     28 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
     29 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     30 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     31 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     32 root      20   0       0      0      0 S   0.0   0.0   0:00.61 kauditd
     34 root      20   0       0      0      0 S   0.0   0.0   0:00.38 khungtaskd
     35 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
```

d. Display memory usage and filesystem disk utilization. (1 mark)

- Df -h is used to filesystem utiliztion

```
[ec2-user@ip-172-31-45-35 ~]$ df -h
Filesystem        Size  Used Avail Use% Mounted on
devtmpfs          4.0M     0  4.0M   0% /dev
tmpfs             457M     0  457M   0% /dev/shm
tmpfs             183M  448K  183M   1% /run
efivarfs          128K  2.7K  121K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p1    8.0G  1.8G  6.2G  23% /
tmpfs             457M     0  457M   0% /tmp
/dev/nvme0n1p128   10M  1.3M  8.7M  13% /boot/efi
tmpfs              92M     0   92M   0% /run/user/1000
```

- Free -h used to memoeryusage

```
[ec2-user@ip-172-31-45-35 ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:           913Mi       197Mi       201Mi       0.0Ki       514Mi       577Mi
Swap:             0B          0B          0B
```

SECTION C: PYSPARK:

Create a PySpark DataFrame

```
C: > Users > ktsre > Desktop > question7.py > ...
1
2   from pyspark.sql import SparkSession
3   from pyspark.sql.functions import col, avg, count, to_date
4
5   spark = SparkSession.builder \
6       .appName("EmployeeDataFrame") \
7       .master("local[*]") \
8       .getOrCreate()
9
10  data = [
11      (101, "Arun", "IT", 75000, "2022-01-15"),
12      (102, "Bharath", "IT", 60000, "2021-03-10"),
13      (103, "Charan", "IT", 85000, "2020-06-20"),
14      (104, "Deepak", "IT", 55000, "2023-02-12"),
15      (105, "Eshwar", "HR", 65000, "2022-07-01"),
16      (106, "Farhan", "HR", 72000, "2021-09-15"),
17      (107, "Ganesh", "HR", 58000, "2024-01-05"),
18      (108, "Harish", "Finance", 90000, "2019-11-25"),
19      (109, "Ishan", "Finance", 68000, "2020-04-18"),
20      (110, "Jay", "Finance", 62000, "2023-08-22"),
21      (111, "Kiran", "Marketing", 58000, "2024-05-14"),
22      (112, "Lokesh", "Marketing", 61000, "2022-12-10"),
23      (113, "Manoj", "Marketing", 57000, "2021-10-30"),
24      (114, "Naveen", "IT", 95000, "2018-02-01"),
25      (115, "Praveen", "HR", 80000, "2020-12-12")
26  ]
27
28  columns = [
29      "EmployeeID", "EmployeeName", "Department",
30      "Salary", "JoiningDate"
31  ]
32
33  df = spark.createDataFrame(data, columns)
34  df = df.withColumn(
35      "JoiningDate",
36      to_date(col("JoiningDate"), "yyyy-MM-dd")
37  )
38
39  print("\n(a) SCHEMA")
40  df.printSchema()
41
42  print("\nFIRST FIVE RECORDS")
43  df.show(5, truncate=False)
44
45  print("\n(b) EMPLOYEES EARNING MORE THAN 60000")
46  df.filter(col("Salary") > 60000).show(truncate=False)
47
48  print("\n(c) ANNUAL SALARY")
49  df = df.withColumn("AnnualSalary", col("Salary") * 12)
50  df.select("EmployeeName", "Salary", "AnnualSalary").show(truncate=False)
51
52  print("\n(d) DEPARTMENT-WISE SUMMARY")
53  df.groupBy("Department").agg(
54      avg("Salary").alias("AverageSalary"),
55      count("*").alias("EmployeeCount")
56  ).orderBy("Department").show(truncate=False)
57
58  print("\n(e) TOP FIVE HIGHEST-PAID EMPLOYEES")
59  df.orderBy(
60      col("Salary").desc(),
61      col("EmployeeID").asc()
62  ).select(
63      "EmployeeID", "EmployeeName", "Department", "Salary"
64  ).show(5, truncate=False)
65
66  spark.stop()
67
```

Question 7 PySpark DataFrame Operations:

a. Display the schema and first five records. (1 mark)

```
(a) SCHEMA
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)
```

b. Filter employees whose monthly salary is greater than ₹60,000. (1 mark)

```
(b) EMPLOYEES EARNING MORE THAN 60000
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Arun        |IT        |75000 |2022-01-15 |
|103       |Charan      |IT        |85000 |2020-06-20 |
|105       |Eshwar      |HR        |65000 |2022-07-01 |
|106       |Farhan      |HR        |72000 |2021-09-15 |
|108       |Harish      |Finance   |90000 |2019-11-25 |
|109       |Ishan       |Finance   |68000 |2020-04-18 |
|110       |Jay         |Finance   |62000 |2023-08-22 |
|112       |Lokesh      |Marketing |61000 |2022-12-10 |
|114       |Naveen      |IT        |95000 |2018-02-01 |
|115       |Praveen     |HR        |80000 |2020-12-12 |
+----------+------------+----------+------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary. (1 mark)

```
(c) ANNUAL SALARY
+------------+------+------------+
|EmployeeName|Salary|AnnualSalary|
+------------+------+------------+
|Arun        |75000 |900000      |
|Bharath     |60000 |720000      |
|Charan      |85000 |1020000     |
|Deepak      |55000 |660000      |
|Eshwar      |65000 |780000      |
|Farhan      |72000 |864000      |
|Ganesh      |58000 |696000      |
|Harish      |90000 |1080000     |
|Ishan       |68000 |816000      |
|Jay         |62000 |744000      |
|Kiran       |50000 |600000      |
|Lokesh      |61000 |732000      |
|Manoj       |57000 |684000      |
|Naveen      |95000 |1140000     |
|Praveen     |80000 |960000      |
+------------+------+------------+
```

d. Group employees by department and calculate the average salary and employee count. (2 marks)

```
(d) DEPARTMENT-WISE SUMMARY
+----------+-----------------+-------------+
|Department|AverageSalary    |EmployeeCount|
+----------+-----------------+-------------+
|Finance   |73333.33333333333|3            |
|HR        |68750.0          |4            |
|IT        |74000.0          |5            |
|Marketing |56000.0          |3            |
+----------+-----------------+-------------+
```

e. Display the five highest-paid employees. (1 mark)

```
(e) TOP FIVE HIGHEST-PAID EMPLOYEES
+----------+------------+----------+------+
|EmployeeID|EmployeeName|Department|Salary|
+----------+------------+----------+------+
|114       |Naveen      |IT        |95000 |
|108       |Harish      |Finance   |90000 |
|103       |Charan      |IT        |85000 |
|115       |Praveen     |HR        |80000 |
|101       |Arun        |IT        |75000 |
+----------+------------+----------+------+
only showing top 5 rows
(.venv) ktsre@KTSReddy-PC:/mnt/c/Users/ktsre/Desktop/spark$
```

Question 8: Data Cleaning and Aggregation

a. Read the CSV file using an appropriate schema and header configuration. (1 mark)

```
(a) ORIGINAL TRANSACTIONS
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product |Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|T001         |C101      |2026-10-01     |Laptop  |1       |55000.0  |
|T002         |C102      |02/10/2026     |Mouse   |2       |500.0    |
|T003         |C103      |2026-10-03     |Keyboard|1       |1500.0   |
|T004         |C104      |2026-10-04     |Laptop  |1       |55000.0  |
|T002         |C102      |02/10/2026     |Mouse   |2       |500.0    |
|T005         |NULL      |2026-10-05     |Mouse   |3       |500.0    |
|T006         |C106      |invalid-date   |Keyboard|2       |1500.0   |
|T007         |C107      |2026-10-07     |NULL    |1       |2000.0   |
|T008         |C108      |2026-10-08     |Monitor |NULL    |12000.0  |
|T009         |C109      |2026-10-09     |Monitor |2       |NULL     |
+-------------+----------+---------------+--------+--------+---------+
```

b. Identify and remove duplicate transactions using a suitable business key. (1 mark)

```
(b) AFTER REMOVING DUPLICATES
+-------------+----------+---------------+--------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product |Quantity|UnitPrice|
+-------------+----------+---------------+--------+--------+---------+
|T001         |C101      |2026-10-01     |Laptop  |1       |55000.0  |
|T002         |C102      |02/10/2026     |Mouse   |2       |500.0    |
|T003         |C103      |2026-10-03     |Keyboard|1       |1500.0   |
|T004         |C104      |2026-10-04     |Laptop  |1       |55000.0  |
|T005         |NULL      |2026-10-05     |Mouse   |3       |500.0    |
|T006         |C106      |invalid-date   |Keyboard|2       |1500.0   |
|T007         |C107      |2026-10-07     |NULL    |1       |2000.0   |
|T008         |C108      |2026-10-08     |Monitor |NULL    |12000.0  |
|T009         |C109      |2026-10-09     |Monitor |2       |NULL     |
+-------------+----------+---------------+--------+--------+---------+
```

c. Handle missing values in required fields and explain your chosen approach. (2 marks)

- the dropna() function removes records containing missing values in the specified required columns.
- This approach is chosen because missing transaction details can lead to incorrect revenue calculations and unreliable analysis.

d.Convert TransactionDate into a date type and handle invalid dates. (1 mark)

```
+-------------+----------+--------+--------+---------+---------------+
|TransactionID|CustomerID|Product |Quantity|UnitPrice|TransactionDate|
+-------------+----------+--------+--------+---------+---------------+
|T001         |C101      |Laptop  |1       |55000.0  |2026-10-01     |
|T002         |C102      |Mouse   |2       |500.0    |2026-10-02     |
|T003         |C103      |Keyboard|1       |1500.0   |2026-10-03     |
|T004         |C104      |Laptop  |1       |55000.0  |2026-10-04     |
+-------------+----------+--------+--------+---------+---------------+
```

e. Calculate revenue using Quantity × UnitPrice. (1 mark)

```
(e) TRANSACTIONS WITH REVENUE
+-------------+----------+--------+--------+---------+---------------+-------+
|TransactionID|CustomerID|Product |Quantity|UnitPrice|TransactionDate|Revenue|
+-------------+----------+--------+--------+---------+---------------+-------+
|T001         |C101      |Laptop  |1       |55000.0  |2026-10-01     |55000.0|
|T002         |C102      |Mouse   |2       |500.0    |2026-10-02     |1000.0 |
|T003         |C103      |Keyboard|1       |1500.0   |2026-10-03     |1500.0 |
|T004         |C104      |Laptop  |1       |55000.0  |2026-10-04     |55000.0|
+-------------+----------+--------+--------+---------+---------------+-------+
```

f. Calculate total revenue by product and display the results in descending order. (1 mark)

```
(f) TOTAL REVENUE BY PRODUCT
+--------+------------+
|Product |TotalRevenue|
+--------+------------+
|Laptop  |110000.0    |
|Keyboard|1500.0      |
|Mouse   |1000.0      |
+--------+------------+
```

Code:

```
print("\n(a) Original data")
df.show(truncate=False)

# (b) Remove repeated transactions
df = df.dropDuplicates(["TransactionID"])

print("\n(b) After removing duplicates")
df.show(truncate=False)

# (c) Remove rows with missing details
df = df.dropna(subset=[
    "TransactionID", "CustomerID", "TransactionDate",
    "Product", "Quantity", "UnitPrice"
])

# Keep only valid quantities and prices
df = df.filter((col("Quantity") > 0) & (col("UnitPrice") > 0))

print("\n(c) After removing missing values")
df.show(truncate=False)

# (d) Convert the dates into a proper date format
df = df.withColumn(
    "NewDate",
    coalesce(
        to_date(try_to_timestamp(col("TransactionDate"), lit("yyyy-MM-dd"))),
        to_date(try_to_timestamp(col("TransactionDate"), lit("dd/MM/yyyy"))),
        to_date(try_to_timestamp(col("TransactionDate"), lit("MM/dd/yyyy")))
    )
)

# Remove rows where the date is invalid
df = df.filter(col("NewDate").isNotNull())

df = df.drop("TransactionDate")
df = df.withColumnRenamed("NewDate", "TransactionDate")

print("\n(d) After cleaning dates")
df.show(truncate=False)

# (e) Find the revenue for each transaction
df = df.withColumn("Revenue", col("Quantity") * col("UnitPrice"))

print("\n(e) Revenue for each transaction")
df.show(truncate=False)

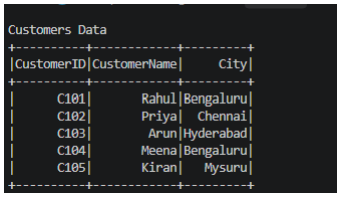
# (f) Find total revenue for each product
print("\n(f) Total revenue by product")

df.groupBy("Product").agg(
    total("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc()).show(truncate=False)

# Close Spark
spark.stop()
```

Question 9: PySpark Joins:

a. Load both datasets into DataFrames. (1 mark)



```
Customers Data
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|      C101|       Rahul|Bengaluru|
|      C102|       Priya|  Chennai|
|      C103|        Arun|Hyderabad|
|      C104|       Meena|Bengaluru|
|      C105|       Kiran|   Mysuru|
+----------+------------+---------+
```

b. Join the datasets to display orders with customer details. (2 marks)

```
Orders with Customer Details
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName|City     |OrderID|OrderAmount|OrderDate |
+----------+------------+---------+-------+-----------+----------+
|C101      |Rahul       |Bengaluru|O101   |5000       |2026-10-01|
|C102      |Priya       |Chennai  |O102   |3000       |2026-10-02|
|C101      |Rahul       |Bengaluru|O103   |2000       |2026-10-03|
|C104      |Meena       |Bengaluru|O104   |4500       |2026-10-04|
+----------+------------+---------+-------+-----------+----------+
```

c.Identify customers who have never placed an order. (1 mark)

```
Customers Without Orders
+----------+------------+---------+
|CustomerID|CustomerName|City     |
+----------+------------+---------+
|C103      |Arun        |Hyderabad|
|C105      |Kiran       |Mysuru   |
+----------+------------+---------+
```

d. Calculate the total order value by city. (1 mark)

```
Total Order Value by City
+---------+---------------+
|City     |TotalOrderValue|
+---------+---------------+
|Chennai  |3000           |
|Bengaluru|11500          |
+---------+---------------+
```

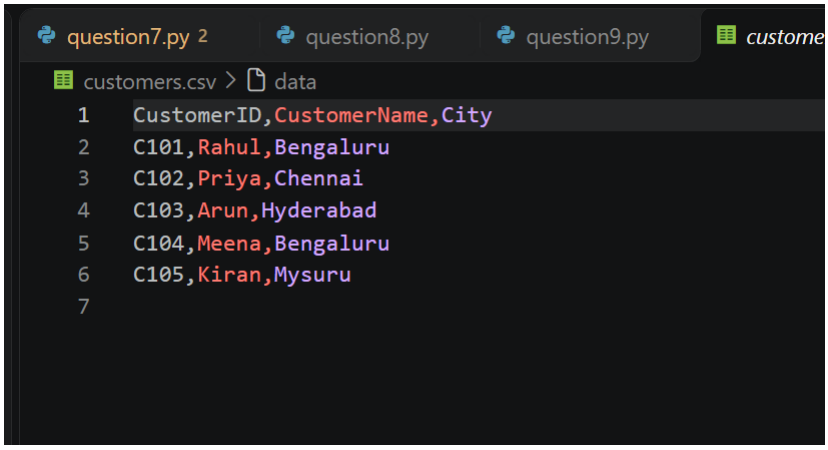
e. Explain when a broadcast join is useful and what conditions make it appropriate. (1 mark)

- A broadcast join is useful when one DataFrame is small enough to fit in the memory of each worker. Spark sends the smaller DataFrame to all workers, reducing data shuffling and improving performance.

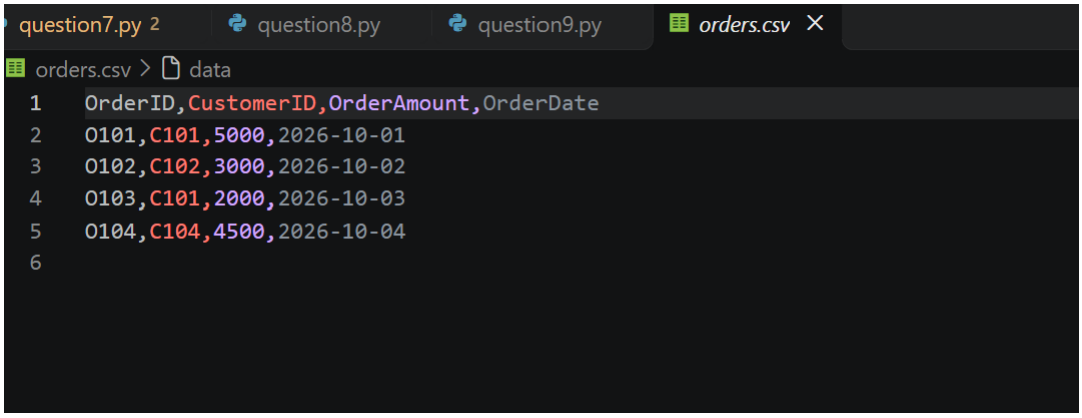
f. f. Explain the difference between transformations and actions, giving one example of each. (1 mark)

- A broadcast join is useful when one DataFrame is small enough to fit in the memory of each worker. Spark sends the smaller DataFrame to all workers, reducing data shuffling and improving performance.

Customer csv :



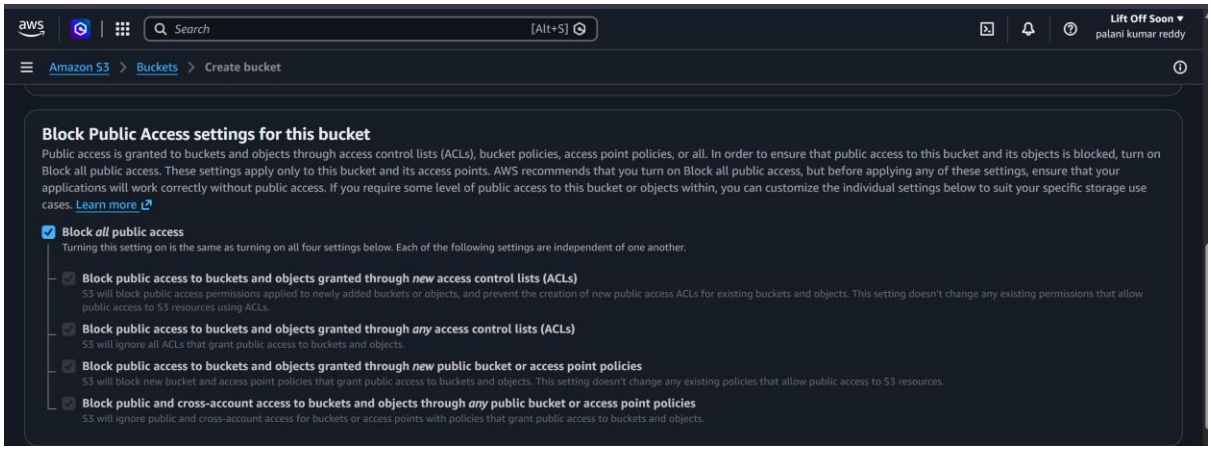
Orders csv:

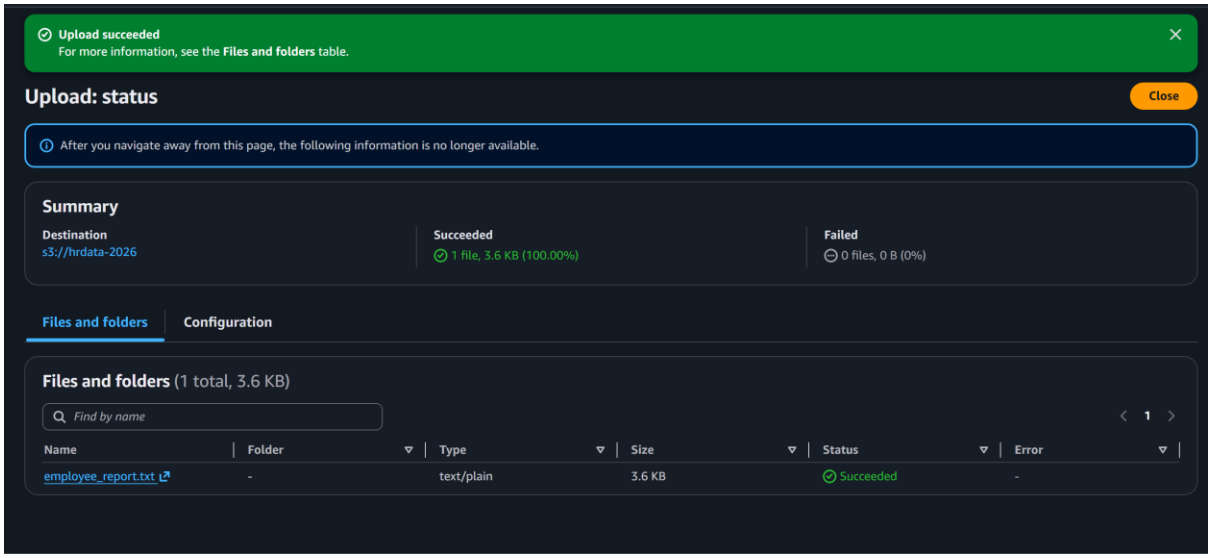


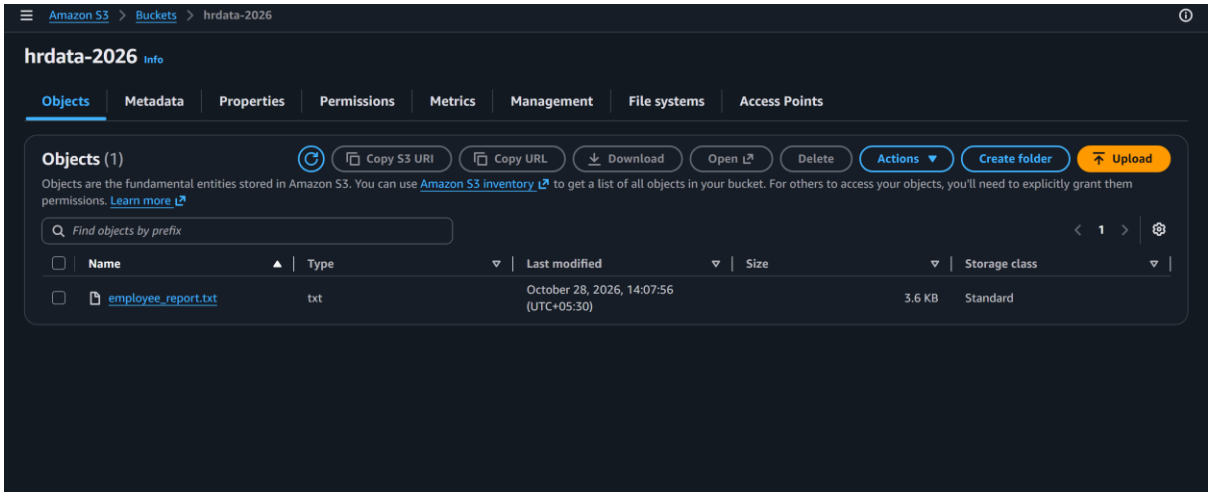
SECTION D: AWS CLOUD SERVICES:

Question 10: S3 and IAM Security:

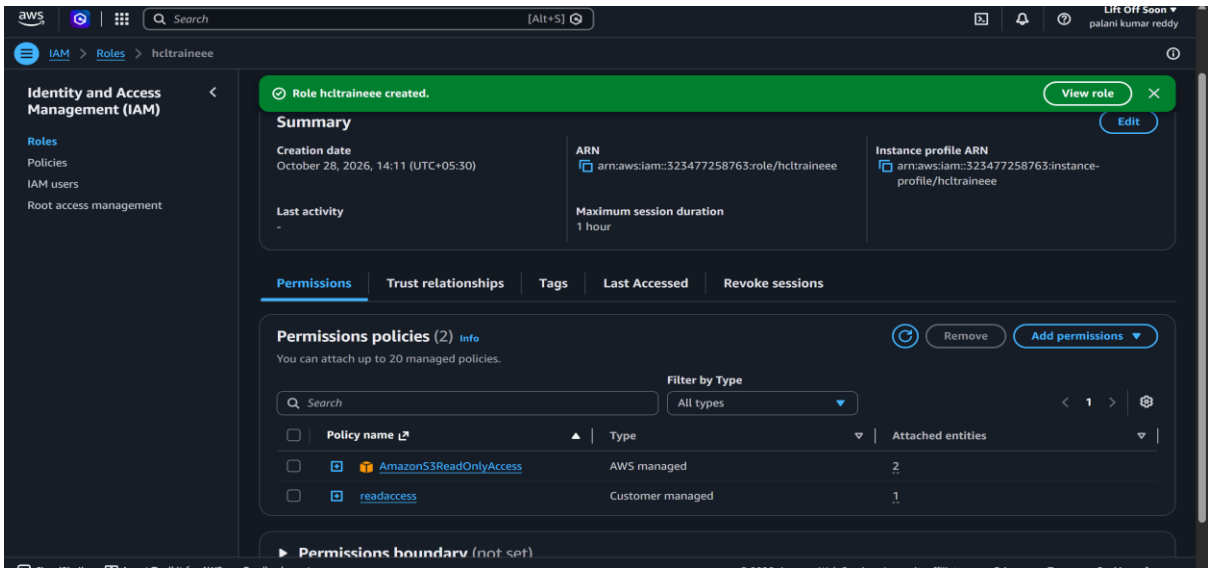
a. Create an S3 bucket with Block Public Access enabled and upload a sample file. (1 mark)

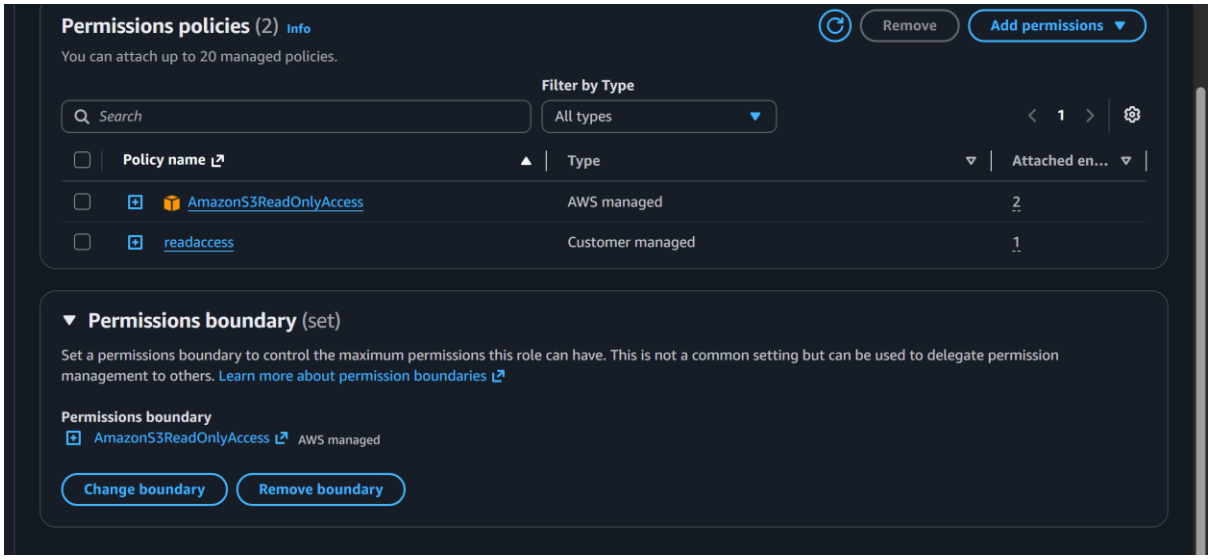






b.Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)





c.Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

- Sign in using an IAM test user that does not have permission to access the bucket.
- Attempt to open or download employee_report.txt.
- Verify that AWS returns Access Denied.

d.Explain how encryption at rest protects the stored data. (1 mark)

- Encryption at rest protects stored S3 data by converting it into an unreadable encrypted form.
- Amazon S3 automatically encrypts all stored data at rest using SSE-S3 (AES-256) by default at no extra cost.

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks):

a. Check the EC2 instance state and status checks. (1 mark):

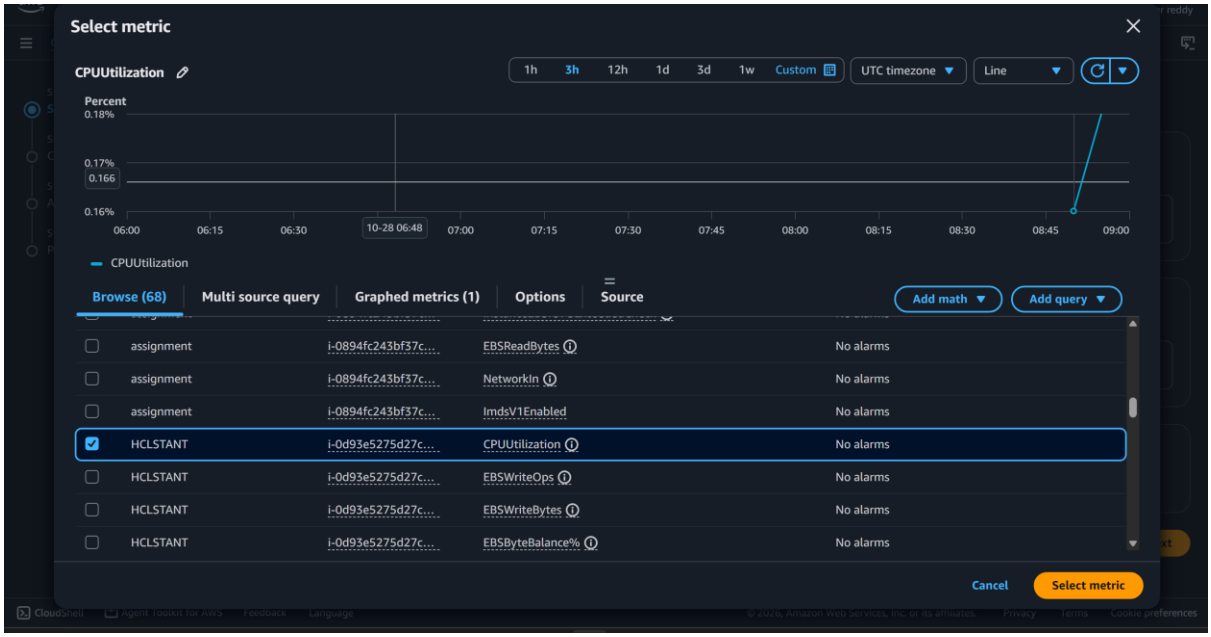


b. Explain how security groups and route tables can affect application connectivity. (1 mark):

- Security groups: Allow or block traffic through specified ports. HTTP uses port 80 and HTTPS uses port 443.
- • Route tables: Direct network traffic. Incorrect routes or a missing internet gateway route can prevent users from reaching the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. (2 marks)

- Open Amazon CloudWatch and select Alarms and Create alarm.
- Select the EC2 instance's CPUUtilization metric.
- Set the CPU utilization greater than 85% and choose an evaluation period of 5 minutes.
- Name the alarm and create it.
- The alaram will get on when it come to the threshold 85% .





d. Identify one method for investigating the root cause of high CPU utilization. (1 mark)

- We'll use the command "top" to investigate the root cause of high cpu utilization it should be done by connecting the ec2 instance to the server.

Question 12: Event-Driven Processing (5 Marks):

(a) Identify a suitable service for executing the processing code. (1 mark)

- AWS Lambda is suitable for running processing code automatically without managing servers.

(b) Explain how an S3 upload can trigger processing. (1 mark)

- Configure an S3 Event Notification to trigger a Lambda function whenever a new file is uploaded to the input bucket.

(c) Explain the IAM permissions needed to read the input file and write the results. (1 mark)

- Give the Lambda execution role permission to read input files using s3:GetObject and write results using s3:PutObject on the required S3 bucket paths only.

(d) Explain how errors and processing logs can be captured. (1 mark)

- Use Amazon CloudWatch Logs to store Lambda execution logs and error messages. CloudWatch can also monitor failures and help troubleshoot problems.

(e) Explain how to prevent duplicate processing or recursive triggering. (1 mark)

- Use a unique file key or event ID to detect duplicate processing.
- Store output files in a separate S3 bucket or a different folder from the input files to prevent the Lambda function from triggering itself repeatedly.

SECTION E: AWS STEP FUNCTIONS:

Question 13: State Machine Design (7 Marks):

a. Identify suitable states for validation, processing, storage, and notification. (2 marks)

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)

- A Choice state checks the validation result. If the file is valid, it routes the workflow to the processing state. If the file is invalid, it routes to a failure state using the `Default` path.

c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)

- Retry: Automatically retries a state when a temporary error occurs, such as a network timeout.
- Catch: Handles permanent or repeated errors by routing the workflow to a failure-handling state, where the error can be logged or a notification sent.

d.Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)

- The state machine uses Task states to invoke AWS Lambda functions or other supported AWS services through service integrations, allowing each service to perform a specific task in the workflow.

e. Explain how execution history helps troubleshoot a failed workflow. (1 mark)

- Execution history records each state's input, output, status, and error messages. It helps identify which state failed and the reason for failure, making troubleshooting easier.

Question 14: Step Functions Implementation (8 Marks):

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary. (3 marks)

```
{
  "Comment": "Sales Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:ap-southeast-2:323477258763:function:ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:ap-southeast-2:323477258763:function:TransformData",
      "Next": "StoreResults"
    },
    "StoreResults": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:ap-southeast-2:323477258763:function:StoreResults",
      "End": true
    }
  }
}
```

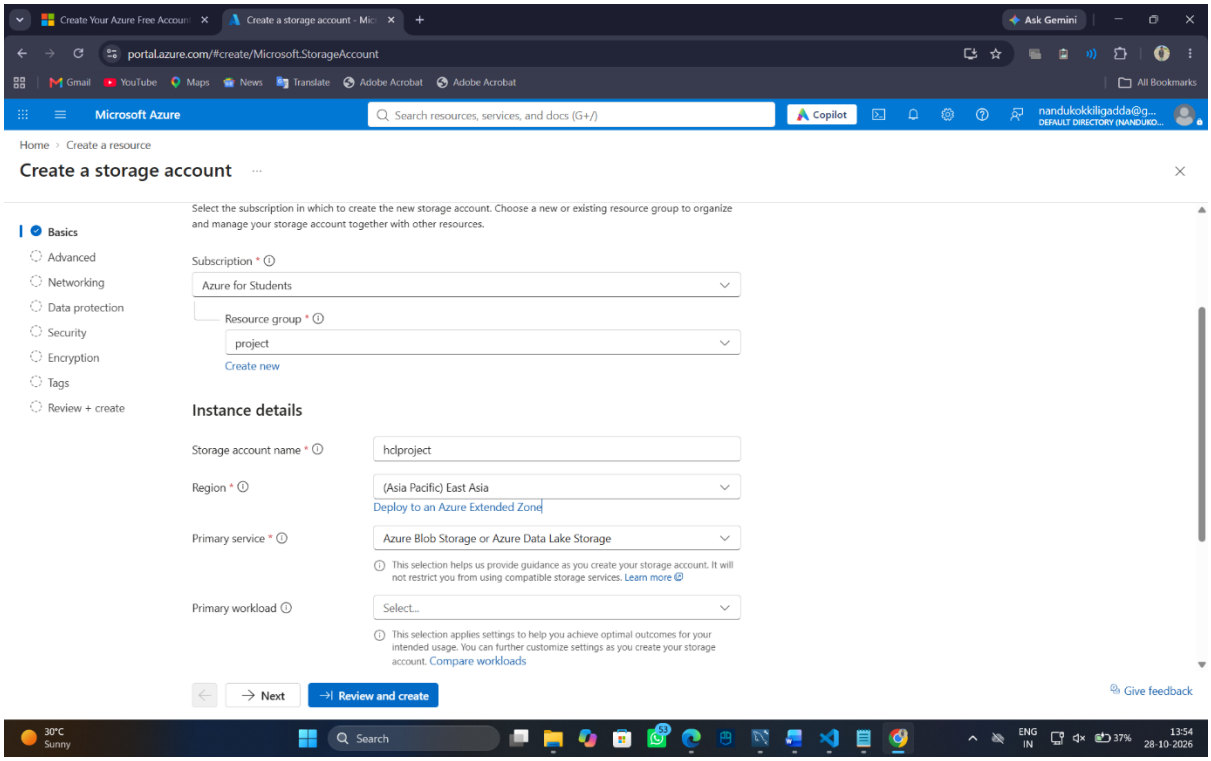
b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts. (2 marks)
    - Explanation:
    - ValidateFile: Validates the input file.
    - TransformData: Transforms the data.
    - StoreResults: Stores the processed results in Amazon S3.
    - Next defines the next task, and End: true marks the workflow's completion.
c. Configure a Catch transition for the loading task that routes to an error-handling state. (1 mark)
    - The Catch block captures errors and routes the workflow to the ErrorHandler state for handling the failure.
d. Explain how data from one task is passed to the next task. (1 mark)

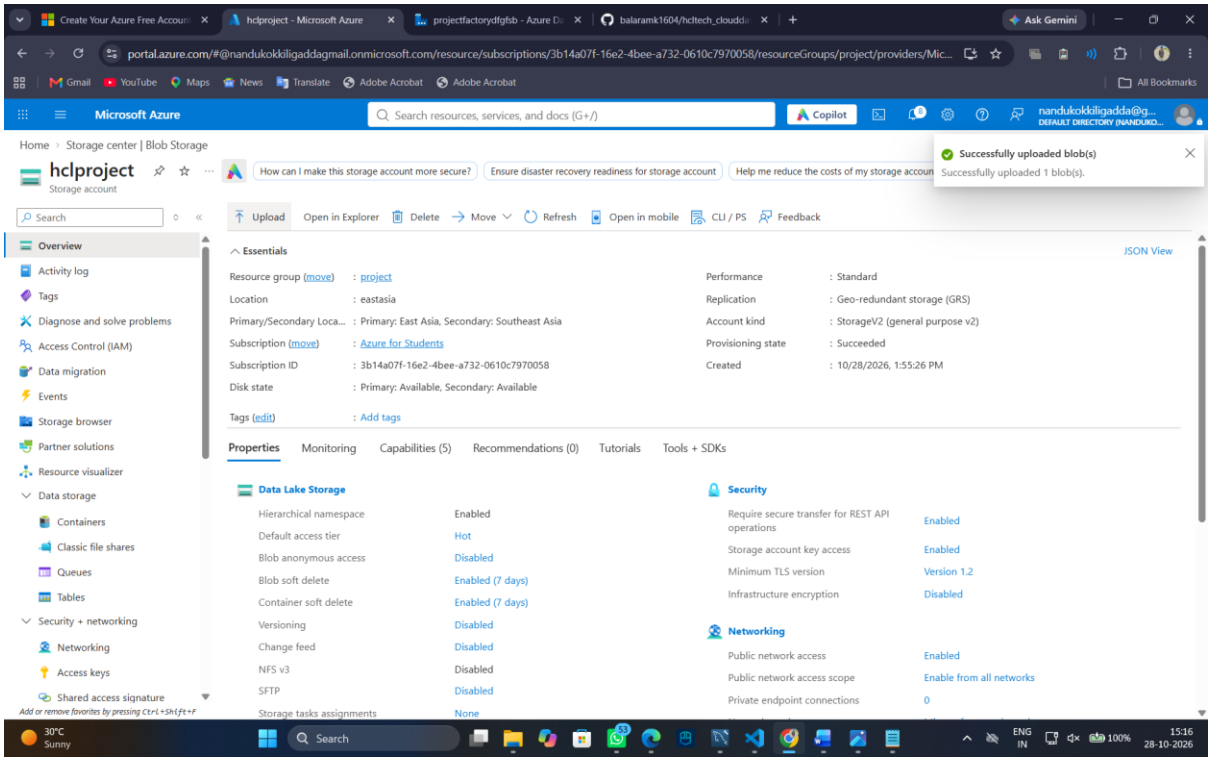
    - Data is passed through the JSON input and output of each state. The output of one task becomes the input of the next task using the Next transition.
e. Explain how you would test successful and failed executions. (1 mark)
    - Start execution in AWS Step Functions with valid input to test success. Provide invalid input or simulate a Lambda error to test failure handling, then check the Execution History to verify the results and error transitions.

SECTION F: MICROSOFT AZURE

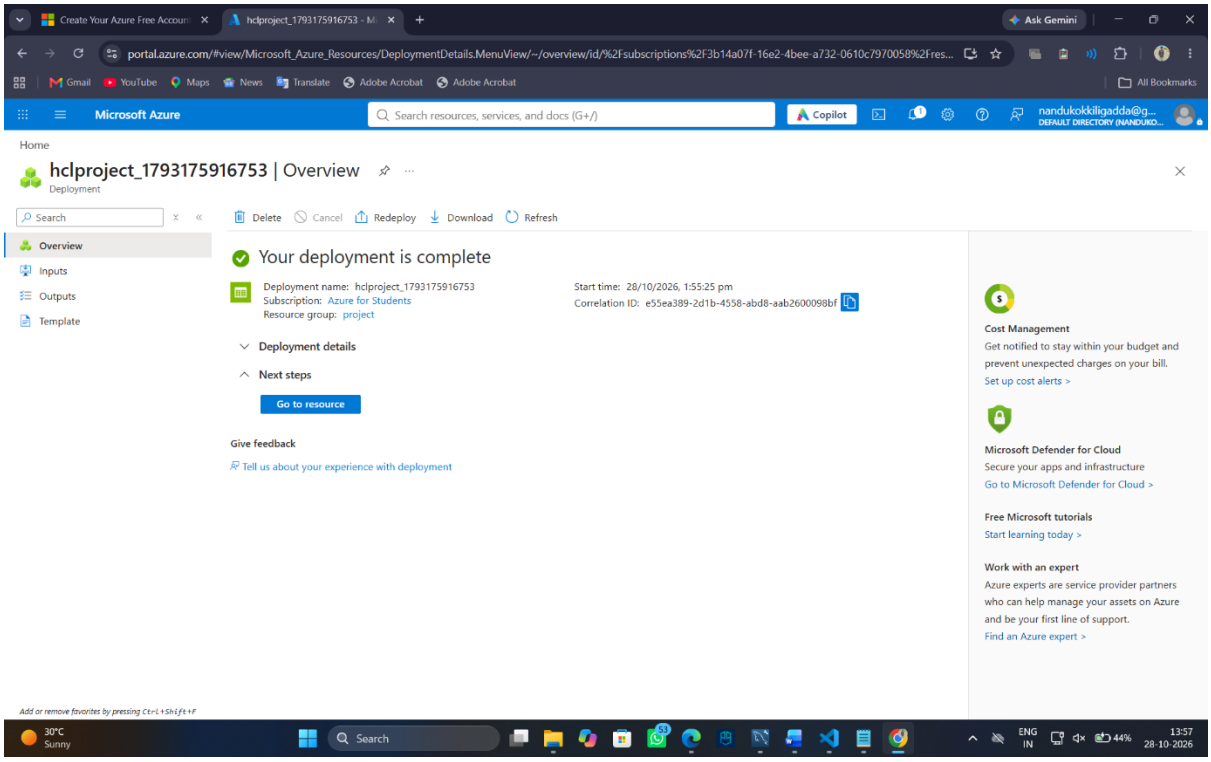
Question 15: Azure Data Lake Storage and Security (5 Marks):

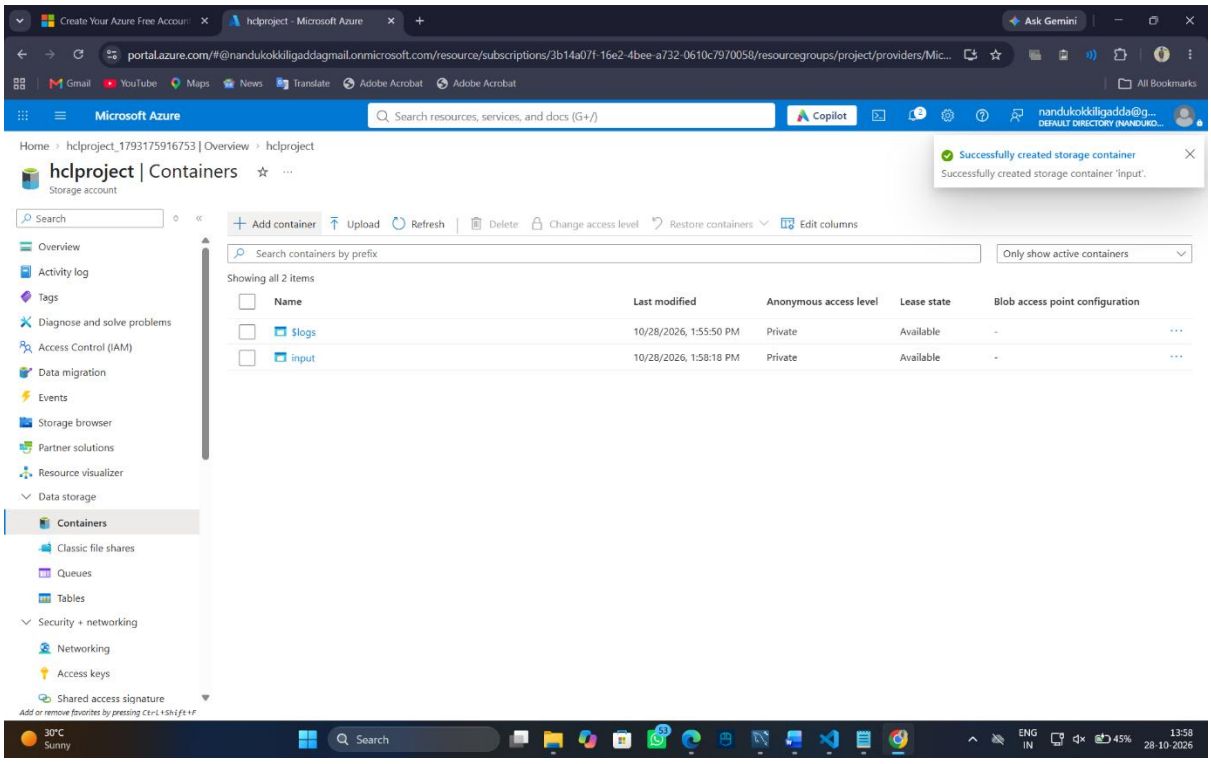
a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2. (1 mark)

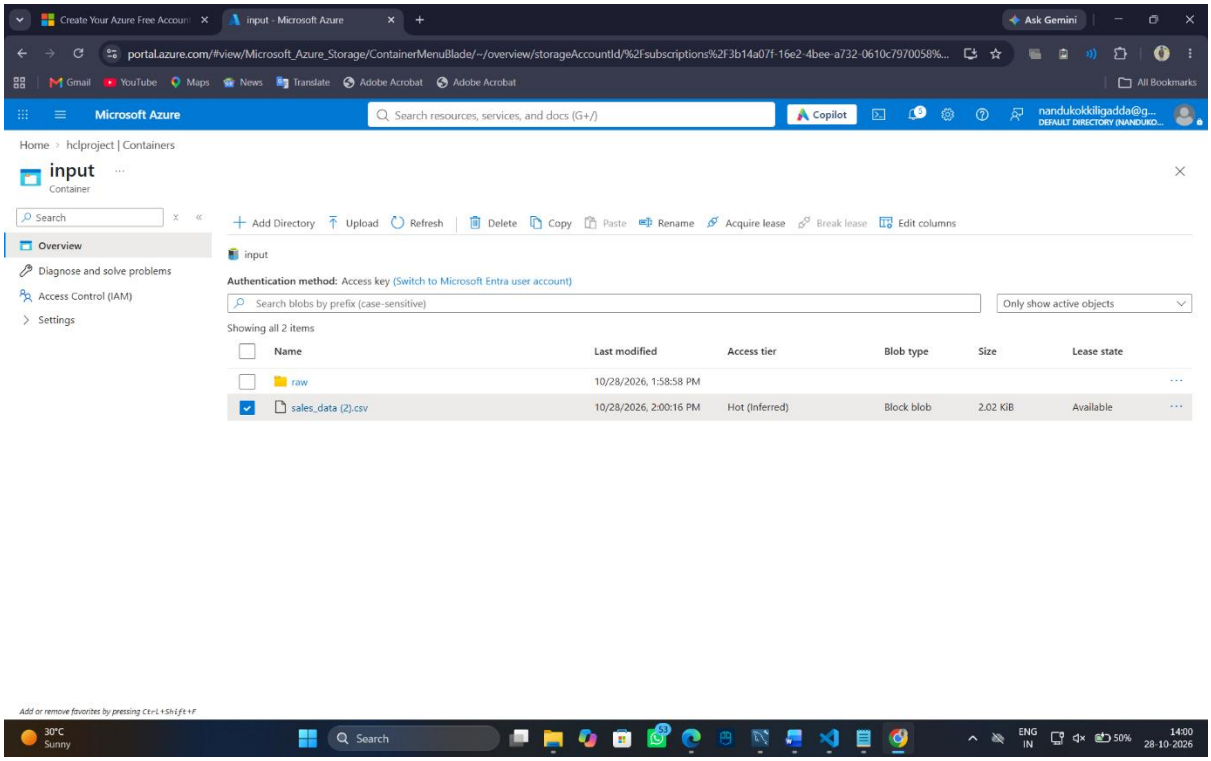




b. Create a filesystem or container and upload a sample dataset. (1 mark)







c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data. (2 marks)

- Microsoft Entra ID provides identity and authentication. Azure role-based access control (RBAC) assigns permissions to users, groups and service principals.
- Suitable roles include:
- Storage Blob Data Reader: Read data without modifying it.
- Storage Blob Data Contributor: Read, write and delete data within the assigned scope

d. Explain how you would verify that unauthorized users cannot access the dataset.

- Assign the least-privilege role at the appropriate storage account, container or supported directory scope. Azure Data Lake Storage Gen2 also supports POSIX-style access control lists (ACLs) for directory and file permissions.

Question 16: Azure VM Troubleshooting

a. **Verify the VM's provisioning and power states**.

- In the Azure Portal, navigate to Virtual Machines and select the VM. Check:
- Provisioning state: Succeeded, Failed or another reported state.
- Power state: Running, Stopped, Deallocated or another reported state.

b.Explain how Network Security Group rules can prevent connectivity.

- NSGs contain inbound and outbound rules that permit or deny network traffic. A missing SSH rule for TCP port 22 or RDP rule for TCP port 3389 can prevent connections. Rules are evaluated according to priority.

c.Explain how subnet and routing configuration affect connectivity.

- An incorrect subnet configuration, user-defined route, route table or next-hop setting can prevent traffic from reaching the VM. Verify the effective routes, subnet association, network interface configuration and any required gateway or peering connections**.**

d.Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

- Use Azure Bastion to connect to the VM over SSH or RDP through the Azure Portal without assigning the VM a public IP address.
- Additional checks:
- Use Network Watcher connection troubleshooting.
- Review NSG effective security rules and effective routes.
- Check VM boot diagnostics and serial console where supported.
- Avoid opening SSH or RDP to the entire internet.

Question 17: Azure Data Integration

.a.Select an appropriate service to orchestrate data movement.

- **Use Azure Data Factory (ADF) to create and orchestrate a data pipeline that automates the movement and processing of daily customer transaction CSV files**

b. Copy the CSV file into Azure Data Lake Storage Gen2 (1 mark)

Create an Azure Data Factory pipeline with a Copy Data activity. Configure the CSV file as the source and Azure Data Lake Storage Gen2 as the destination. Set up the required linked services and authentication to transfer the file securely.

c. Describe how to organize files by ingestion date. (1 mark)

ADF dynamic content can generate the folder path using the ingestion date, making files easy to locate and manage.

d. Handle malformed records and missing values (1 mark)

Use Azure Databricks or Azure Data Factory Mapping Data Flows to validate and transform the data. Send malformed records to a separate error folder, handle missing values by applying suitable defaults or null values, and remove duplicates where necessary.

e. Monitor pipeline execution and validate data completeness (1 mark)

Use Azure Data Factory Monitor to track pipeline runs, failures, and execution duration. Validate data completeness by comparing source and destination record counts, checking required fields, and logging rejected records. Configure Azure Monitor alerts for pipeline failures.